arXiv:2510.09720v1 [cs.CL] 10 Oct 2025

# Preference-Aware Memory Update for Long-Term LLM Agents

**Haoran Sun, Zekun Zhang, Shaoning Zeng**


## Abstract

One of the key factors influencing the reasoning capabilities of LLM-based agents is their ability to leverage long-term memory. Integrating long-term memory mechanisms allows agents to make informed decisions grounded in historical interactions. While recent advances have significantly improved the storage and retrieval components—e.g., by encoding memory into dense vectors for similarity search or organizing memory as structured knowledge graphs—most existing approaches fall short in memory updating. In particular, they lack mechanisms for dynamically refining preference memory representations in response to evolving user behaviors and contexts. To address this gap, we propose a Preference-Aware Memory Update Mechanism (PAMU) that enables dynamic and personalized memory refinement. By integrating sliding window averages (SW) with exponential moving averages (EMA), PAMU constructs a fused preference-aware representation that captures both short-term fluctuations and long-term user tendencies. We conduct experiments on five task scenarios of the LoCoMo dataset, and the results show that our mechanism can significantly improve the output quality of LLM in five baselines, validating its effectiveness in long-term conversations.


## Introduction

Large Language Model (LLM) agents exhibit strong autonomous decision-making capabilities across a wide range of tasks, particularly excelling in open-domain question answering (Yao et al. 2024; Huang et al. 2024a; DeepSeek-AI 2025). In long-term dialogue scenarios, effective reasoning and decision-making often require integrating past interactions, making internal memory mechanisms essential (Zhang et al. 2024a,b). These mechanisms aim to emulate human-like cognitive memory by retaining prior conversational context, enabling the agent to retrieve relevant information and generate context-aware, personalized responses. The design and adaptation of such memory systems are thus critical to the agent's performance in complex, temporally extended tasks (Li et al. 2024; Guo et al. 2024; Sun et al. 2025; Sun and Zeng 2025).

The most basic memory approach concatenates prior dialogues with the current prompt, but this method is constrained by the LLM's finite context window, limiting its effectiveness in prolonged interactions (Jin et al. 2024; Gu et al. 2024). To address this, recent studies have explored more sophisticated architectures: MemoryBank (Zhong et al. 2024) encodes past information into dense vectors and retrieves memories via similarity search ; MemGPT (Packer et al. 2023) introduces a hierarchical OS-inspired memory system that combines limited-context attention with external memory storage, yet suffers from a trade-off between retrieval accuracy and efficiency; MemInsight (Salama et al. 2025) enhances memory representation by autonomously extracting structured key-value attributes; and A-MEM (Xu et al. 2025), inspired by the Zettelkasten method, dynamically constructs evolving knowledge graphs for self-organizing memory.

Despite these advances, existing systems predominantly focus on memory storage and retrieval, often overlooking a crucial aspect: how to adaptively and continuously update memory in response to evolving user behavior during long-term interactions (Wu et al. 2025; Huang et al. 2024b). In real-world deployment, users are non-stationary—their intents, preferences, and goals shift over time. Without dynamic memory updating, agents risk relying on outdated or misaligned information, leading to degraded performance and user trust.

To bridge this gap, we propose a Preference-Aware Memory Update Mechanism that enables LLMs to perceive, adapt to, and respond in alignment with evolving user preferences. At its core is a novel Preference Change Perception Module, which combines a sliding window average and an exponential moving average (EMA) to construct a dual-perspective user preference representation—capturing short-term behavioral shifts while robustly modeling long-term trends. We further introduce a formalized change detection signal, triggered by the deviation between short- and long-term estimates, to guide when and how memory updates should occur. This allows for interpretable and controllable adaptation in response to preference drift. Notably, our mechanism is highly modular and model-agnostic: it requires no fine-tuning or architectural modification and can be seamlessly integrated into existing memory-augmented LLM frameworks.

## Related Work

To enhance the long-term reasoning capabilities of LLM agents, various memory systems have been proposed.

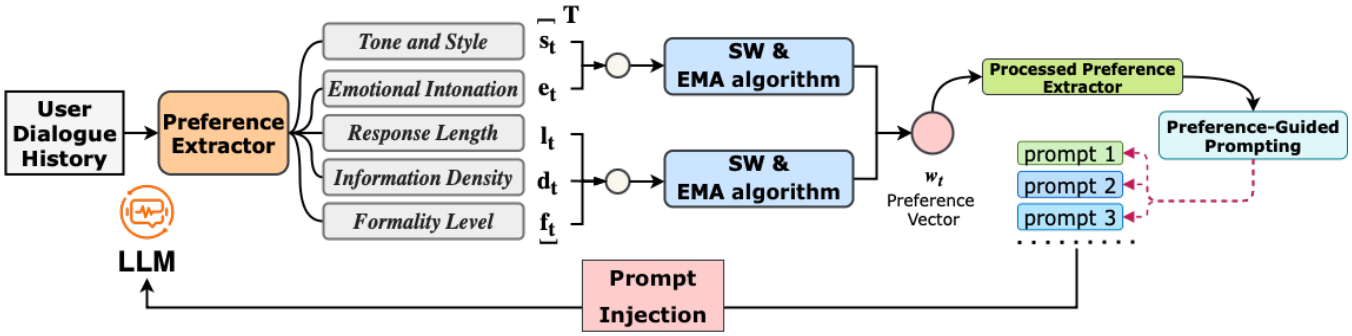


Figure 1: **Illustration of PAMU method.** PAMU extracts user preferences from dialogue, models short- and long-term trends via SW and EMA, detects preference shifts, and updates the prompt to guide personalized generation.

ReadAgent (Lee et al. 2024) segments and compresses documents into key-point memories for retrieval-augmented reading comprehension. MemGPT (Packer et al. 2023) uses OS-inspired virtual memory management, combining hierarchical memory with external storage via dynamic function calls. SCM (Wang et al. 2023) enables agents to autonomously decide when and how to access memory through a controller-stream-agent framework. MemoryBank (Zhong et al. 2024), grounded in the Ebbinghaus forgetting curve, supports memory storage, retrieval, and update for user-aware personalization. A-MEM (Xu et al. 2025), inspired by Zettelkasten, organizes memory as evolving, self-linked knowledge notes. MemInsight enhances memory representation by extracting structured attributes for more accurate semantic retrieval (Salama et al. 2025).

While these approaches have advanced memory modeling in LLMs—especially in storage, retrieval, and organization—they largely assume static user behavior. In practice, user preferences and goals evolve dynamically. However, existing systems lack mechanisms to adaptively track and update memory in response to such changes. This highlights a critical gap: the need for a dynamic, preference-aware memory update mechanism that supports long-term personalization in LLM agents.

## Methodology

In this section, we introduce our Preference-Aware Memory Update (PAMU) mechanism.

### Preference Extractor

The system constructs a user preference vector $\mathbf{P} = \{p_1, p_2, \ldots, p_D\}$ by extracting multidimensional preference signals from multi-turn interactions between the user and the model. Each dimension $p_d$ represents a specific user preference type, such as tone style, response length, emotional tone, information density, and degree of formality. After each dialogue turn, the system updates the preference vector by analyzing user feedback and linguistic features. Specifically:

- **Tone Style.** A RoBERTa encoder with a multi-class classification head is employed to analyze the stylistic features of user utterances. The model produces a probability distribution over predefined tone categories. The category with the highest probability and its score are concatenated into a tuple to represent the tone dimension.
- **Response Length.** This is measured by the number of tokens generated by the model. The average response length over the past K turns is computed and normalized to the [0, 1] range to form the length dimension.
- **Emotional Tone.** An emotion classification model identifies the dominant emotional categories from both user and assistant utterances. A probability vector over predefined emotional classes is extracted, and the class with the highest probability is used, along with its score, to represent the emotional tone dimension.
- **Information Density.** The system leverages an OpenIE model to extract structured (subject, predicate, object) triples from the assistant's responses. Each triple is treated as an atomic information unit. The number of extracted triples per turn is treated as the count of knowledge points. The information density $ID_t$ of the response at turn t is defined as:

$$ID_t = \frac{K_t}{L_t} \tag{1}$$

  Among them, $K_t$ represents the number of triples sampled in the t-th round, and $L_t$ represents the total number of words in the response of that round. This ratio measures the average amount of information carried by each word, reflecting the compactness of language use and the degree of knowledge density.
- **Degree of Formality.** A pretrained formality classification model is employed to evaluate the assistant's response, yielding a normalized formality score within the range [0, 1], where 0 indicates fully colloquial (spoken) language and 1 denotes fully formal (written) language. This score is directly used as the value for the formality dimension.

Accordingly, for each dialogue turn, the system extracts a five-dimensional user preference vector:

$$\mathbf{p}_t = (s_t, l_t, e_t, d_t, f_t) \tag{2}$$

Here, $s_t$ and $e_t$ denote tuples containing the predicted category index and its probability for tone style and emotional

Algorithm 1: Preference-Aware Dialogue Generation

**procedure** GENERATERESPONSE($H_t$, $x_t$) ▷ History and current user input
  $p_t \leftarrow$ EXTRACTPREFERENCES($H_t, x_t$) ▷ $p_t = (s_t, l_t, e_t, d_t, f_t)$
  **for all** $d \in$ {tone, length, emotion, density, formality} **do**
    $SW_t[d] \leftarrow$ Mean($p_{t-W+1:t}[d]$) ▷ Sliding window average
    $EMA_t[d] \leftarrow \beta \cdot EMA_{t-1}[d] + (1-\beta) \cdot p_t[d]$
    $w_t[d] \leftarrow \lambda \cdot SW_t[d] + (1-\lambda) \cdot EMA_t[d]$
  **end for**
  $desc \leftarrow$ FORMATPREFERENCE($w_t$) ▷ Natural language preference prompt
  $prompt \leftarrow$ "Respond in style: " + $desc$ + "\n" + $x_t$
  $y_t \leftarrow$ LLM.generate($prompt$)
  **return** $y_t$
**end procedure**
**function** EXTRACTPREFERENCES($H_t$, $x_t$)
  $s_t \leftarrow$ ToneClassifier($x_t, H_t$) ▷ Categorical: RoBERTa-based
  $l_t \leftarrow$ Normalize(MeanLength($r_{t-K:t-1}$))
  $e_t \leftarrow$ EmotionAnalyzer($x_t, H_t$)
  $d_t \leftarrow$ InfoDensity($r_{t-1}$) ▷ Triple/token ratio
  $f_t \leftarrow$ FormalityDetector($x_t$)
  **return** $(s_t, l_t, e_t, d_t, f_t)$
**end function**
**function** FORMATPREFERENCE($w_t$)
  **return** [Tone: Label($w_t$[tone]), Emotion: Label($w_t$[emotion]),
    Density: Quantize($w_t$[density]), Length: Quantize($w_t$[length]),
    Formality: Quantize($w_t$[formality])]
**end function**

tone, respectively; $l_t$, $d_t$, and $f_t$ are normalized scalar values representing response length, information density, and formality. This vector is then fed into the Preference Shift Detector, which models the temporal dynamics of user preferences using a combination of a sliding window mechanism and Exponential Moving Average (EMA). This enables the system to detect both gradual drifts and abrupt shifts in preferences, and to determine whether the model's response strategy requires adaptation or fine-tuning to better align with evolving user intent.

## Preference Change Perception Mechanism

Following the extraction of multi-dimensional user preference vectors, a Preference Dynamics Perception Module is employed to model behavioral shifts and enable personalized response adaptation. This module integrates Sliding Window (SW) averaging with Exponential Moving Average (EMA) to continuously update preference estimates at each dialogue turn, thereby guiding the response generator toward controlled, user-aligned outputs.

Specifically, we uniformly represent user preference vectors in the form of:

$$\boldsymbol{p}_t = \left[p_t^{(1)}, p_t^{(2)}, \ldots, p_t^{(D)}\right] \tag{3}$$

Among them, D represents the number of preference dimensions. Each dimensional preference value $p_t^{(d)}$ may be a continuous variable (such as response length, information density, formality level) or a categorical variable (such as tone style, emotional intonation). For categorical variables, we use the tuple $(c_t^{(d)}, q_t^{(d)})$ to represent, where $c_t^{(d)}$ is the category index and $q_t^{(d)}$ is the categorical probability distribution.

**Dynamic Modeling of Continuous Preference Dimensions.** For continuous preference dimensions (length, information density, and Degree of formalization), we define a sliding window of length W to calculate the sliding average preference value at the current time t.

$$\mathrm{SW}_t^{(d)} = \frac{1}{W} \sum_{i=t-W+1}^{t} p_i^{(d)} \tag{4}$$

Among them, $\mathrm{SW}_t^{(d)}$ is the sliding window average of the preference in the $d$-th dimension at time $t$; $W$ is the sliding window length (the number of historical rounds used to calculate the average); $p_i^{(d)}$ represents the preference value in the $d$-th dimension of the $i$-th round; $\sum_{i=t-W+1}^{t}$ denotes the cumulative operation on the preference values from the $(t-W+1)$-th round to the $t$-th round within the window.

Meanwhile, Exponential Moving Average (EMA) is introduced to enhance the memory capacity for long-term trends. Let EMAt(d) denote the exponential average of preference dimension d at time t, then its update formula is:

$$\mathrm{EMA}_t^{(d)} = \beta \cdot \mathrm{EMA}_{t-1}^{(d)} + (1-\beta) \cdot p_t^{(d)} \tag{5}$$

Among them, $\beta \in (0,1)$ is the decay coefficient, which controls the degree of influence of historical preferences on the current estimate. SW is more sensitive to recent preference changes, while EMA is used to slowly track long-term trends.

After the combination of the two, the fused perception vector is defined as:

$$\hat{w}_t^{(d)} = \lambda \cdot \mathrm{SW}_t^{(d)} + (1-\lambda) \cdot \mathrm{EMA}_t^{(d)} \tag{6}$$

Among them, $\lambda \in [0,1]$ controls the weight proportion of the sliding window and exponential average. This fusion strategy can flexibly adapt to the fast-changing and slow-changing characteristics in user preferences.

**Dynamic Modeling of Categorical Preference Dimensions.** For categorical dimensions (tone style and emotional intonation), we represent the preference of each round as $(c_t^{(d)}, q_t^{(d)})$, which is the currently most likely category and its corresponding probability distribution. We perform sliding average and exponential average on the category probability distribution vectors respectively: